



OPEN-SOURCE EV DIAGNOSTIC TOOL

A modular, software-defined platform for comprehensive
vehicle system analysis



Project Objectives



- **Modular Architecture:**

Develop a modular, software-defined diagnostic platform.



- **Open-Source Accessibility:**

Remove barriers to EV diagnostic development.



- **Hardware Independence:**

Provide a virtualized environment (vCAN/vECU) to simulate vehicle behavior.



- **Standard Diagnostic :**

Standardize UDS protocol implementation for modern EVs.



Methodology & Approach

- **Defining the software architecture:**
Deciding the data flow and overall architecture for the tool.
- **Modular Design:**
Separating UI from Protocol logic to allow for independent updates.
- **Software-Defined First:**
Prioritizing simulation over physical hardware dependencies during early development.
- **Scalable Open-Source Framework:**
Designing the software as an open-source, modular platform that allows developers to easily add new ECUs, UDS services and diagnostic features.



Implemented Solution

Modular Diagnostic Tool

→ Developed a layered diagnostic application with independent GUI, protocol, communication, and translation modules.

Used globally accepted automotive language

→ Implemented ISO 14229 UDS key services: 0x10, 0x22, 0x19, 0x14, 0x3E over ISO-TP and CAN for standardized vehicle diagnostics.

Diagnostic Data Translation

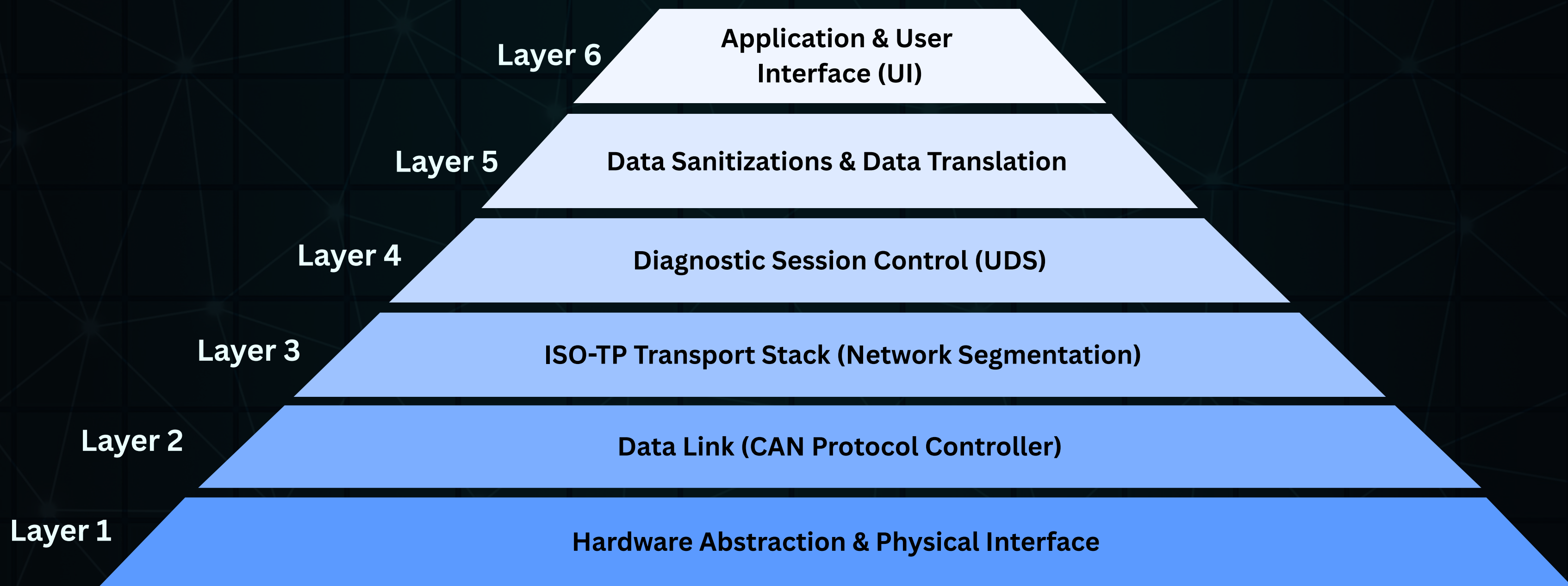
→ Converted raw SIDs, DID values and Negative Response Codes (NRCs) into meaningful diagnostic information using database-driven decoding.

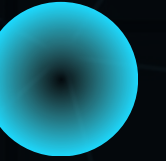
Simulated ECU

Made a simulated ECU for the testing purpose of the Diagnostic tool.



System Architecture





Core Capabilities

Raw UDS Commands : Send custom UDS diagnostic requests.

Configurable CAN IDs : Modify Tx/Rx IDs for ECU communication.

Session Monitoring: Display the present diagnostic session.

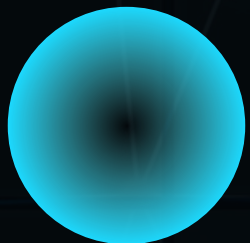
Decoded Responses : View both raw hex and human-readable results.

Communication History : Show all requests and responses with clearing option.

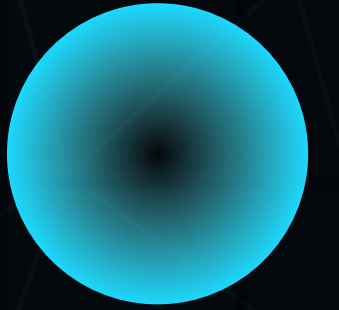
Modular Architecture : Easily extensible to services, DIDs, and hardware support.

Request Timeout Handling : Wait for ECU responses within a configurable timeout.

Automatic Tester Present : Automatically transmit 0x3E to keep the ECU active .



Future Scope



- **Shift to Real hardware:**
Integrating with real EV.
- **OEM Protocol Integration:**
Implementing manufacturer-specific command sets for expanded diagnostic depth.
- **Database-Driven DTC Management:**
Integrate standard Diagnostic Trouble Codes (DTCs) into an Excel database, enabling dynamic decoding.
- **Low-Cost Hardware Interface:**
cost-effective hardware interface using microcontrollers (e.g., Arduino or ESP32) with a CAN transceiver as alternative to the commercial CAN adapter.



Thank you!!